

GROUNDWORK

Technical Roadmap, Architecture & AI Implementation Specification

Prepared for: Jeffrey Prepared by: FreelanceComm Agency Classification: Confidential

This document expands on our GROUNDWORK technical understanding brief with a complete, transparent build plan: the system architecture, request-level data flows, database model, matching engine mechanics, AI implementation detail for every AI-assisted feature, the youth safeguarding architecture, monitoring strategy, and a phased development roadmap with sprint-level deliverables.

1. Executive Summary

GROUNDWORK is a multi-sided football opportunity ecosystem connecting adult players, youth players (under parental supervision), clubs, and administrators. It is built around three cooperating systems — the Player Platform, the Club Recruitment Platform, and the Administrative & Safeguarding Platform — unified by a deterministic matching engine and a set of narrowly-scoped AI features.

This document is the technical companion to our earlier understanding brief. Where that document established what we are building, this document establishes exactly how: the service architecture, the data model, the step-by-step request flows for the platform's core actions, the AI pipeline design, the safeguarding controls enforced at the data layer, and a realistic phase-by-phase delivery roadmap.

SCOPE

This specification covers architecture and request flow (Section 2–3), data model (Section 4), matching engine (Section 5), AI implementation (Section 6), safeguarding architecture (Section 7), monitoring (Section 8), and the full development roadmap with sprint breakdown (Section 9–10).

2. System Architecture Overview

2.1 Layered Architecture

The platform is organised into five layers. Each layer has a single responsibility and communicates with adjacent layers only through defined interfaces (REST/JSON over HTTPS internally, signed URLs for media), which keeps the system testable in isolation and allows individual services to scale independently.

Layer	Responsibility	Technology
Client Layer	Player portal, club portal, and admin panel UIs; form validation and optimistic UI state	Next.js, TypeScript, Tailwind CSS
API Layer	Authentication, request validation, business logic, RBAC enforcement	Node.js, Express, JWT
Processing Layer	Matching engine, scoring engine, AI orchestration, video processing	BullMQ workers, Redis, Claude API, FFmpeg
Data Layer	System of record for all structured data	PostgreSQL (primary), Redis (cache/queues)
Storage & Delivery	Media storage and global delivery	AWS S3, CloudFront CDN

2.2 Core Services

Business logic is grouped into independently deployable services behind the Express API gateway. This separation lets the AI and video-processing workloads scale differently from the low-latency auth and search paths.

Service	Owns
Auth Service	Registration, login, JWT issuance/refresh, RBAC roles, session revocation
Player Service	Adult & youth profile CRUD, video metadata, opportunity feed, applications
Club Service	Club onboarding, verification status, search/filter, shortlists, trials
Matching Engine	Weighted scoring, precomputed match table, recalculation triggers
AI Orchestration Service	Squad gap analysis, video tagging, recommendation generation via Claude API
Safeguarding Service	Guardian consent workflow, club youth-verification, consent audit log
Payments Service	Stripe checkout, subscription/boost webhooks, idempotency
Notification Service	Transactional and push notifications via OneSignal
Admin Service	Moderation queue, verification review, GDPR export/erasure, audit logs

3. Architecture Flow — Core Request Paths

The following flows describe exactly what happens, step by step, for the platform's four most important interactions. These are the flows we will implement first, and they double as the acceptance criteria for Sprint 1–2 of Phase 1.

3.1 Player Registration & Profile Scoring Flow

1. Player submits registration form (Next.js) — email/password or OAuth via Auth Service; JWT issued with role = player.
2. Player completes profile builder: position, physical attributes, playing history, availability, club/team history.
3. On save, the API writes the profile row to PostgreSQL inside a transaction and publishes a profile.updated event to a Redis-backed BullMQ queue.
4. A scoring worker consumes the event, runs the deterministic scorePlayer() function (Section 5), and writes the result to the precomputed player_scores table with a version stamp.
5. If the score changed materially, a rescore-matches job is enqueued to refresh any club-facing match rows referencing this player (mirrors the stale-write protection pattern in Section 5.2).
6. Player is redirected to their dashboard, which reads the freshly computed score and an opportunity feed filtered to their profile.

3.2 Club Search & Matching Flow

7. Club applies filters (position, age band, region, level) in the Club Portal search UI.
8. API Layer validates the club's verification status and, for youth filters, confirms an active youthVerified flag with a non-expired renewal date.
9. Query executes against the precomputed match_results table (indexed on club_id, position, age_band, score DESC) rather than scanning full player records — this keeps search latency independent of total player count.
10. Results return a denormalised snapshot per player (name, position, age band, headline stats, thumbnail) so the list renders without a second round-trip.
11. Opening a full profile issues a separate authorised read against the player record; RBAC middleware checks the club's verification flags before returning youth data.
12. A background worker checks the match_results TTL and refreshes any stale rows asynchronously — the club is never blocked waiting on a rescore.

3.3 Video Upload & AI Tagging Flow

13. Client requests a signed S3 upload URL from the API; the request is rejected server-side if the player already holds the maximum clip count — the UI limit is not the enforcement point.
14. Client uploads directly to S3 using the signed URL; upload progress drives the Next.js progress UI.
15. An S3 event triggers a processing job: FFmpeg validates duration/format, transcodes to a delivery-optimised bitrate, and generates a thumbnail.
16. The AI Orchestration Service samples key frames from the transcoded clip and calls the Claude API with a structured prompt to generate descriptive tags (e.g. dominant foot, playing style cues, position indicators visible in footage).
17. Tags are stored against the video record as suggestions, not facts — they are surfaced to the player for review before appearing on the public profile, and never feed the deterministic scoring engine.
18. CloudFront serves the processed clip and thumbnail; the raw upload is deleted after successful transcode.

3.4 Youth Registration & Guardian Consent Flow

19. Age entered at registration under 18 routes the account into the youth flow automatically; the profile is created with consentStatus = pending and is not searchable.
20. Guardian name, email, and relationship are collected and stored on the youth profile record — never on the player-facing record.
21. A signed, time-limited consent token is emailed to the guardian via the Notification Service, linking to a dedicated consent portal.

22. The guardian reviews a plain-language data notice and submits consent; the API validates the token server-side and writes consentGrantedAt plus an immutable entry to the consent_audit table.
23. Only once consent is granted does the Safeguarding Service flip searchable = true, making the youth profile visible to clubs holding an active youth-verification flag.
24. Consent can be withdrawn at any time through the same portal; withdrawal immediately removes the profile from search and triggers scheduled deletion of associated video clips per the safeguarding policy in Section 7.

4. Data Model

PostgreSQL is the system of record. Core tables below are simplified to their scoring- and safeguarding-relevant columns; full schemas (constraints, indexes, migrations) will be delivered as part of Sprint 1.

4.1 players (adult)

Column	Type — Notes
id	uuid — primary key, matches auth user id
display_name	text
date_of_birth	date
positions	text[] — primary + secondary
skill_ratings	jsonb — per-attribute 1–10 ratings
availability	jsonb — training days / times
region	text — indexed for proximity filtering
profile_completeness	numeric(3,2) — recalculated on write
searchable	boolean — player opt-in to club search
updated_at	timestampz — used for stale-write guard

4.2 players_youth

Inherits all adult columns plus a guardian relationship and consent state, held in a separate table so that row-level security policies can isolate youth data independently of the adult table.

Column	Type — Notes
guardian_id	uuid — guardian account reference
guardian_email	text — consent audit trail
consent_status	enum — pending / granted / withdrawn
consent_granted_at	timestampz
age_group	text — e.g. U14, U16, U18

4.3 clubs

Column	Type — Notes
id	uuid — primary key
name	text
verified_adult	boolean
verified_youth	boolean — requires safeguarding certificate + admin approval
youth_verification_expiry	timestampz — annual renewal enforced
subscription_tier	enum — free / standard / premium
stripe_customer_id	text

4.4 match_results (precomputed)

Column	Type — Notes
club_id / player_id	uuid — composite key
match_score	numeric(4,3) — primary sort field
score_breakdown	jsonb — per-dimension contribution, for transparency UI
scored_at / ttl_expiry	timestampz — drives background refresh
position / age_band / region	text — denormalised, indexed for filtered search

4.5 consent_audit

Append-only table: every consent action (granted, withdrawn, re-granted) is written as a new row with a timestamp, hashed IP, and the version of the privacy notice agreed to. No client role — including the guardian's own account — has UPDATE or DELETE privileges on this table; only an internal admin service role does, for statutory-retention cleanup.

5. Matching Engine

5.1 Scoring Dimensions

Consistent with our earlier brief, the score is deterministic and rule-based — AI is never in the scoring path. Weights are stored in a configuration table so they can be tuned without a code deploy, and every change is versioned.

Dimension	Weight	Calculation
Position compatibility	30%	Exact match = 1.0, adjacent position = 0.5, incompatible = 0
Distance / region	15%	Decay function from club's preferred radius
Playing level	20%	Normalised comparison of player level vs. club's target tier
Player score / attributes	20%	Weighted average of self-reported and coach-verified attributes
Availability overlap	10%	Ratio of overlapping training/trial availability
Visibility / boost status	5%	Flat bonus while an active boost is live

5.2 Precomputation & Recalculation

Scores are computed once, on write, and cached in `match_results` — never recalculated at query time. This is what keeps club search latency flat as the player base grows (see Section 3.2). When a player updates a scoring-relevant field, a background job recalculates their score against every club that currently has them in a result set, guarded by an optimistic-concurrency check on `updated_at` so that a slower, stale recalculation can never overwrite a fresher one.

DETERMINISM

The scoring function takes player data, club preferences, and the current config as explicit inputs and returns a pure result — no wall-clock reads, no randomness, no floating-point drift between environments. This makes the engine unit-testable and auditable, which matters both for player trust and for any future dispute over why a match score was what it was.

6. AI Integration — Implementation Detail

To restate the core principle plainly: AI is not used for player scoring. Scoring stays rule-based and transparent end-to-end (Section 5), so every match score can be explained by its inputs and weights alone. AI is scoped to four assistive features that sit entirely outside that scoring path — three built in Phase 1, one planned for Phase 2 — each implemented as its own pipeline with a defined input, a constrained Claude API call, a defined output schema, and a human review step before anything AI-generated becomes visible to another user.

DESIGN PRINCIPLE

Every AI feature on GROUNDWORK follows the same shape: structured input in, structured (never free-form) output out, and a visible "AI-assisted" label with an accept/edit/remove step owned by the person the content is about. This is what keeps AI helpful without letting it quietly become a second, opaque scoring system.

6.1 Squad Gap Analysis

Purpose: help a club see, at a glance, which positions their squad is short on and which player profiles on the platform would close that gap — without the club having to manually cross-reference squad sheets against search filters.

Stage	Detail
Trigger	Club opens the Squad Gap Analysis panel, or a scheduled weekly refresh runs for clubs with active recruitment status.
Input	Structured form data only — squad counts per position, target formation, target playing level. No free text is accepted into the prompt.
Model call	Claude API call with a fixed system prompt scoped strictly to squad-composition reasoning, given the club's squad JSON and formation as input.
Output schema	A ranked list of { position, gap_severity, rationale } objects — no numeric "score" is generated, only a severity label (e.g. Critical / Moderate / Minor) and a one-line rationale.
Surfaced as	A ranked gaps panel on the club dashboard, each gap linking to a pre-filtered search (using the existing deterministic search from Section 3.2) for players in that position — the AI identifies the need, the deterministic engine finds the players.
Cost control	Response is cached against a hash of the squad JSON; an unchanged squad does not trigger a repeat API call.

6.2 Video Auto-Tagging

Purpose: reduce the manual effort of describing match footage by generating a first draft of tags and a short summary the player can review, edit, or discard — never a permanent, unchallengeable label on their profile.

Stage	Detail
Trigger	Fires automatically once FFmpeg transcode completes (Section 3.3), in the same background worker pool.
Input	A small number of representative frames sampled at fixed intervals from the transcoded clip, plus basic metadata (declared position, clip duration).
Model call	Claude API vision call with a prompt constrained to observable, descriptive output only — camera framing, general area of play, visible footwork or movement pattern. The prompt explicitly forbids inferring skill level, assigning a rating, or making comparative judgements.
Output schema	A short list of descriptive tags plus a one-to-two sentence neutral summary — both plain strings, no scores or rankings.
Surfaced as	Suggested tags/summary attached to the video record, greyed out until the player accepts them; the player can edit or delete any tag before or after publishing.
Guardrail	For youth accounts, this job checks consent_status = granted before it is ever enqueued (Section 6.5).

6.3 Player Improvement Recommendations

Purpose: give players specific, actionable suggestions for improving their visibility and opportunity flow — the kind of advice a recruiter might give informally, generated from the player's own structured profile data.

Stage	Detail
Trigger	Scheduled job identifies profiles below a completeness or engagement threshold — e.g. missing availability, no video uploaded, sparse skill ratings, low recent search appearances.
Input	The player's own structured profile-gap data (which fields are missing, how completeness compares to peers in the same position/region) — never another player's personal data, and no free text beyond the player's own entries.
Model call	Claude API call with a prompt scoped to generating specific, actionable next steps from the gap data provided — not general career advice.
Output schema	A short checklist of { action, reason } pairs, e.g. "Add your training availability — clubs filter by this in 70% of searches."
Surfaced as	A checklist on the player's own dashboard, delivered via a Notification Service nudge; never shown to clubs or other players.

6.4 Future AI Features — Phase 2

Two further AI features are scoped for Phase 2, once the core platform and the three Phase-1 AI features above have real usage data to design against:

- Tactical Fit Explanations — given a club's playing style (already captured for matching) and a shortlisted player's profile, generate a short, human-readable explanation of why the deterministic match score came out the way it did. This is explicitly an explainability layer on top of the existing score, not a new scoring signal.
- Career Pathway Insights — using a player's history and typical progression patterns for their age group and level, surface general guidance on realistic next steps (e.g. trial types worth pursuing at their stage). Framed as informational context, never as a guarantee or a ranking against other players.

Both will follow the same structured-input / structured-output / human-review pattern established in Phase 1, and both will be scoped in detail with the client before implementation begins.

6.5 AI Safety & Cost Guardrails

- Every AI call uses a fixed, versioned system prompt and a JSON-constrained output schema — no open-ended generation is exposed to end users, and prompts are version-controlled alongside the codebase.
- AI output is always labelled as AI-assisted in the UI and is reviewable/editable/removable by the profile or club owner before it becomes visible to anyone else.
- AI never writes to the `player_scores` or `match_results` tables — those are populated exclusively by the deterministic engine in Section 5, and this is enforced by giving the AI Orchestration Service no write permission on either table at the database role level.
- Requests are rate-limited per club/player and results are cached against a content hash, so re-opening a panel or re-viewing a clip doesn't trigger a repeat API call — this keeps Claude API spend predictable and roughly proportional to new content, not to traffic.
- No youth player data is sent to any AI pipeline until guardian consent is granted; the video-tagging and recommendation jobs both check `consent_status` before enqueueing, and this check is repeated at execution time in case consent was withdrawn while the job was queued.

7. Youth Safeguarding & Compliance Architecture

Safeguarding is enforced at the data-access layer, not just in the UI, so that a club without the correct verification cannot see youth data through any route — including a direct API call.

7.1 Access Control

- Row-level security policies on `players_youth` require either an active, non-expired `verified_youth` flag on the requesting club, the guardian's own account, or an internal admin service role.
- Youth verification (safeguarding certificate + manual admin approval) is stored in a separate admin-only table, never readable by club-facing API roles directly.
- Annual re-verification is enforced by comparing `youth_verification_expiry` against the current request time on every read — not just at login.

7.2 Consent Lifecycle

The consent flow described in Section 3.4 is backed by the append-only `consent_audit` table (Section 4.5). Withdrawal cascades automatically: the profile is delisted from search immediately, associated video clips are queued for deletion within 24 hours, and the underlying record is retained in a non-discoverable state only for the statutory minimum period before permanent erasure.

GDPR

Admins can action full data export or erasure requests from the Admin Panel; erasure workflows respect the same statutory-retention window applied to consent withdrawal, and every action is written to the platform audit log.

8. Monitoring, Security & Reliability

Tool	Purpose
Sentry	Error tracking and stack traces across API, workers, and frontend
PostHog / Mixpanel	Product analytics — funnel tracking for registration, search, applications
Structured logging	JSON logs from all services; log-based alerts for safeguarding and payment events
Uptime checks	Synthetic probes against critical endpoints (auth, search, checkout) on a short interval
Queue dashboards	BullMQ/Redis queue depth and failure-rate monitoring for scoring and AI jobs

Alerting Priorities

- P1 (immediate page): auth outage, payment webhook failures, youth-data access-control errors.
- P2 (business hours): elevated error rate on scoring/matching jobs, AI pipeline failures, consent-email delivery failures.
- P3 (tracked): queue depth warnings, non-critical latency regressions.

9. Development Roadmap — Phase 1 (MVP)

Phase 1 is broken into four two-week sprints (eight weeks total), each ending in a working, demonstrable build on a staging environment for your review before we proceed to the next. Work runs in parallel across a Frontend track, a Backend track, and an Integration track within each sprint, rather than sequentially — this is what keeps an eight-week timeline realistic for a platform of this scope.

Sprint	Focus	Deliverables
Sprint 1	Foundation	Auth service, RBAC, database schema & migrations, CI/CD pipeline, Next.js app scaffold for all three portals, staging environment live.
Sprint 2	Player & Club Core	Player profile builder, club onboarding & verification, deterministic scoring engine with test suite, precomputed match table.
Sprint 3	Search, Video & Safeguarding	Club search & filtering UI, video upload/transcode pipeline, AI tagging & squad gap analysis, guardian consent flow end-to-end.
Sprint 4	Payments, Hardening & Launch	Stripe integration with webhook idempotency, monitoring/alerting stack live, security review, production deploy, handover documentation.

9.1 Sprint 1 (Weeks 1–2) — Foundation

Goal: a deployable skeleton of the whole system — every portal reachable, every environment provisioned, nothing built on shaky ground later.

- Backend: PostgreSQL schema and migrations for players, clubs, matches, and consent tables (Section 4); Auth Service with JWT issuance/refresh and RBAC role definitions (player / guardian / club / admin).
- Frontend: Next.js project scaffolding for the Player Portal, Club Portal, and Admin Panel, sharing a common component library and design tokens from day one to avoid visual drift later.
- Integration: CI/CD pipeline (build, test, deploy) targeting a staging environment; Sentry and structured logging wired in from the first deploy, not bolted on later.

SPRINT 1 HANDOFF

You can log in as a player, a club, and an admin on staging, and see three distinct, correctly-scaffolded portals — the foundation every later sprint builds on.

9.2 Sprint 2 (Weeks 3–4) — Player & Club Core

Goal: the platform's two primary user types can fully represent themselves in the system, and the deterministic engine that everything else depends on is live and tested.

- Backend: deterministic `scorePlayer()` implementation with the unit-test suite covering every dimension boundary from Section 5.1; precomputed `match_results` table and the recalculation job with the stale-write guard (Section 3.1).
- Frontend: multi-step player profile builder (positions, skills, availability); club onboarding and verification-request flow.
- Integration: `profile.updated` event pipeline connecting profile saves to the scoring worker via BullMQ/Redis.

SPRINT 2 HANDOFF

A player can build a complete profile and see their own computed score breakdown; a club can register and see verification status — the end-to-end scoring loop from Section 3.1 works on real data.

9.3 Sprint 3 (Weeks 5–6) — Search, Video & Safeguarding

Goal: the two most distinctive parts of GROUNDWORK — intelligent club search and youth safeguarding — are fully working together, since neither is meaningful without the other in production.

- Backend: club search API against `match_results` with the composite indexes from Section 3.2; video upload token-gating, FFmpeg transcode pipeline, and the AI Orchestration Service (squad gap analysis and video auto-tagging from Section 6.1–6.2).

- Frontend: club search/filter UI with ranked results; video upload widget with progress and tag-review UI; guardian consent portal (Section 3.4).
- Integration: row-level security policies enforcing youth data isolation (Section 7.1); consent_audit append-only logging wired to every consent action.

SPRINT 3 HANDOFF

A club can search and filter for adult players end-to-end; a youth player can register, a guardian can grant consent through the portal, and the youth profile only becomes visible to youth-verified clubs — safeguarding is provably enforced, not just described.

9.4 Sprint 4 (Weeks 7–8) — Payments, Hardening & Launch

Goal: everything built in Sprints 1–3 is production-ready, monitored, and safely deployable — not just working on staging.

- Backend: Stripe checkout for boosts/subscriptions with idempotent webhook handling (mirroring the pattern in the reference architecture — processed-event table with a create()-based duplicate guard).
- Integration: Sentry alert thresholds, uptime checks on auth/search/checkout, queue-depth monitoring on the scoring and AI job queues (Section 8).
- Hardening: security review of RBAC and row-level security policies, load-testing club search at target profile volumes, production environment provisioning with a mandatory approval gate before any production deploy.
- Handover: production deploy, admin runbook, and full handover documentation covering every service in Section 2.2.

SPRINT 4 HANDOFF

GROUNDWORK is live in production, monitored, and documented — with payments, safeguarding, matching, and AI features all working together as one system rather than four separate demos.

10. Phase 2 & Phase 3

Phases 2 and 3 are scoped at a feature level now so you can see the full trajectory of the platform, and will each be broken into their own sprint-level roadmap — like Section 9 above — once Phase 1 is live and we can plan the next phase against real usage data rather than assumptions.

10.1 Phase 2 — Reach & Depth

Focus: extend GROUNDWORK from a web platform clubs and players visit into one they carry with them, and deepen the AI layer now that Phase 1 has produced real profile and match data to design against.

- Native mobile applications for players and clubs — sharing the existing Express API and RBAC model from Phase 1, so no backend rework is required, only a new client.
- Tactical Fit Explanations and Career Pathway Insights (Section 6.4), built on the same structured-input/structured-output pattern as the Phase 1 AI features.
- Analytics enhancements — cohort tracking and recruitment-funnel reporting for clubs, built on the PostHog/Mixpanel instrumentation already live from Phase 1 (Section 8).

10.2 Phase 3 — Scale & Growth

Focus: tooling for power-user clubs managing recruitment at volume, and growth mechanics to expand the platform's network effects.

- Advanced recruitment tooling — bulk shortlist workflows and comparative player analytics, built on top of the existing match_results and shortlist data model rather than a new data layer.
- Additional automation — proactive AI-generated outreach suggestions for clubs, and expanded scoring configurability for admins to tune weights per league or region.
- Growth features — referral mechanics and academy/college partnership tooling, extending the existing club and admin RBAC roles rather than introducing new ones.

11. Next Steps

25. A short kickoff call to confirm scoring weights, RBAC role definitions, and any branding direction for the three portals.
26. We provision the staging and production environments and share access by the end of Sprint 1.
27. We share a draft of the guardian consent notice for your review alongside any legal input you'd like to provide.
28. You receive the first staging build at the end of Sprint 1 and can begin testing the foundation immediately.
29. We proceed sprint by sprint, with a working build to review at the end of each one before moving forward.